

Tigress ve Çeşitlendirme

CEN429 Güvenli Programlama — Hafta 14

Dr. Öğr. Üyesi Uğur CORUH · 18.12.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	1	Kaynaktan kaynağa gizleme · Tigress · lisans · temel akış
2	2	Dönüşüm aileleri · 9. hafta eşlemesi · dönüşüm hattı · adım adım örnek
3	3–5	Çeşitlendirme · ölçme · sınıf içi akış · derleme hattı (S15) · proje

Kısa tarihçe — kaynaktan kaynağa gizleme ve çeşitlendirme

- **1993** — Cohen: **çeşitlendirme** fikri (aynı işlev, farklı binary)
- **1997** — Collberg vd. gizleme taksonomisi; aynı yıl Forrest vd. ayrı kökten **çeşitlendirme**
- **2002** — Nacula vd., CIL: Tigress'in üzerine kurulduğu altyapı
- **~2012–13** — **Tigress**: C için kaynaktan kaynağa + sanallaştırma + çeşitlendirme
- **2015** — **Obfuscator-LLVM**: derleyici tabanlı gizleme
- **2016–17** — Banescu vd. Tigress+KLEE ile **dayanıklılığı ölçer**

Ana kural: **dayanıklılık** ↔ **maliyet**; koruma varlığın değeriyle orantılı seçilir.

Bu hafta nereye oturuyor?

- **9. hafta:** gizleme kurallarını **el ile** öğrendik
- **11. hafta:** anahtar için whitebox
- **Bu hafta (14):** aynı kuralları **araçla, otomatik** ve **çeşitlendirilmiş** uygulamak → Tigress

Üçü bir bütün; bugün otomasyon adımı.

Öğrenme çıktısı

Bu hafta **ÖÇ.3** (binary uygulama korumaları) üstünedir.

Sonunda yapabileceğiniz:

- Kaynaktan kaynağa gizlemeyi açıklamak
- Bir **dönüşüm hattı** kurmayı tarif etmek
- Gizlemeyi **çeşitlendirip ölçmek**
- Gizlemeyi **derleme hattına** koymak (S15)

Baştan bir hatırlatma

9. haftanın ana kuralı bu hafta da geçerli:

Gizleme kırılmazlık vermez, **maliyet** yükseltir.

Tigress bir **sihir değil**; el ile yaptığımızı otomatikleştirir ve **çeşitlendirmeyi** kolaylaştırır.

Önceki haftalardan gelenler

- **Kaynak kod, derleyici, binary dosya** — insanın yazdığı program metninin derleyiciyle makine koduna çevrilip çalıştırılabilir hale gelmesi (**Hafta 9**)
- **Kod gizleme (obfuscation)** — davranışı değiştirmeden kodu anlaşılması zor kılan, saldırı maliyetini artıran karşı önlem (**Hafta 9**)
- **9. haftanın gizleme kuralları (K-01–K-12)** — opak yüklem, aritmetik kodlama, kontrol akışı düzleştirme, dize kodlama, değişken bölme, sanallaştırma gibi el ile uygulanan kurallar; bu hafta bunları Tigress'in dönüşümleriyle tek tek eşliyoruz (**Hafta 9**)
- **CFG (kontrol akışı grafiği)** — temel blokları düğüm, geçişleri kenar yapan şema (**Hafta 9**)
- **Sembolik yürütme** — program yollarını matematiksel kısıt olarak çözen otomatik analiz (ör. KLEE) (**Hafta 9**)
- **Çeşitlendirme (diversification)** — aynı kaynaktan davranışça eş, yapıca farklı binary dosyalar üretme fikri; bu hafta Tigress'in `--Seed` bayrağıyla otomatikleştiriyoruz (**Hafta 9**)
- **CI ve derleme hattı** — her kod değişikliğinde otomatik derleme/test adımlarını çalıştıran sistem (**Hafta 4**)
- **Sürüm kimliği ve özet (hash) değeri** — bir yazılımın hangi binary/kaynak/özet üçlüsüyle dağıtıldığını gösteren kayıt; bu hafta buna bir de tohum alanı ekliyoruz (**Hafta 1**)

Bu haftanın kavramları

Her terim gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

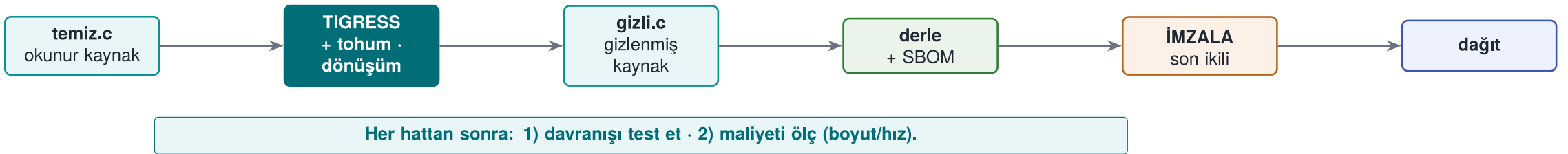
Kavram	Nerede
Kaynaktan kaynağa (source-to-source) gizleme, Tigress ve lisansı	Bölüm 1
Dönüşüm (--Transform) ve hedef seçimi (--Functions)	Bölüm 1
Dönüşüm aileleri ve 9. hafta eşlemesi	Bölüm 2
Dönüşüm hattı (pipeline) ve adım adım maliyet artışı	Bölüm 2
Çeşitlendirme aracı: tohum (--Seed)	Bölüm 3
Gizleme ve çeşitlendirmeyi ölçmek	Bölüm 3
Sınıf içi yedi adımlık akış	Bölüm 4
S15 derleme ve dağıtım hattı, dönem projesi	Bölüm 5

Kaynaktan kaynağa (source-to-source)

- Girdi: C kaynağı. Çıktı: **yine C kaynağı** — ama gizlenmiş.
- Sonra normal derleyicinizle derlenir.

Kaynaktan kaynağa gizleme hattı (S15)

imza EN SON: önce gizle, sonra imzala



Dönüşüm (transform) nedir?

- **Dönüşüm:** kaynağa uygulanan tek bir gizleme işlemi (ör. düzleştirme).
- Araç birçok dönüşüm sunar; siz hangisini, nereye seçersiniz.

Dönüşüm hattı (pipeline)

- **Hat:** birden çok dönüşümün **sırayla** uygulanması.
- Her dönüşüm bir öncekinin çıktısına uygulanır.
- Sıra önemlidir.

Tohum (seed) nedir?

- **Tohum:** rastgeleliği yöneten bir başlangıç sayısı.
- Aynı dönüşüm + farklı tohum = **farklı** gizlenmiş çıktı.
- Çeşitlendirmenin anahtarı budur.

CLI (komut satırı) nedir?

- **CLI (Command-Line Interface):** komutları yazarak çalıştırdığınız arayüz.
- Tigress bir CLI aracıdır: `tigress --Transform=... dosya.c`.

Birim testi (unit test)

- **Birim testi:** bir fonksiyonun doğru çalıştığını otomatik denetleyen küçük test.
- Gizlemeden **sonra** aynı testler geçmeli (davranış korunmalı).

1. Kaynaktan kaynağa gizleme ve Tigress

El ile gizlemenin üç sorunu

9. haftada kuralları **el ile** uyguladık. Öğreticiydi ama:

1. **Hataya açık** — davranışı bozabilir
2. **Bakımı zor** — kaynak okunamaz olur
3. **Çeşitlendirilemez** — her kopyayı elle farklılaştıramazsınız

Çözüm: araca yaptır

- Gizlemeyi bir **araç** yapsın.
- Siz **okunur** kaynağı koruyun.
- Gizleme, derleme hattında **otomatik** bir adım olsun.

Kaynaktan kaynağa fikri

El ile gizleme vs araçla gizleme

9. haftada el ile yaptığımızı 14. haftada araç yapıyor

EL İLE

her sürümde tekrar yaz
kaynak okunamaz hale gelir
hata riski yüksek
çeşitlendirilemez

ARAÇLA (Tigress)

her derlemede otomatik
kaynak TEMİZ kalır
tutarlı ve ölçekli
tohumla çeşitlendirilir

Kaynağı siz korursunuz; gizleme derleme hattının bir adımıdır.

Bakım maliyeti **okunur kaynakta** kalır; dağıtılan kaynak gizli.

Sıralı dönüşümler — şema

Dönüşüm hattı: sıra önemlidir

her dönüşüm bir öncekinin çıktısı üzerinde çalışır



Yanlış sıra bazı dönüşümleri etkisizleştirir ya da maliyeti gereksiz artırır.

Tigress nedir?

- Arizona Üniversitesi'nden Collberg ve ekibinin geliştirdiği **kaynaktan kaynağa C gizleyici/çeşitlendirici**.
- ix. haftadaki el ile kuralların **otomatik** karşılığı.

Tigress · özellikler

- Çok platform: Linux, macOS, Windows, Android
- Mimariler: Intel, ARM, WebAssembly
- Derleyiciler: GCC, Clang, MSVC
- Güncel sürüm: **v4**

Lisans (önemli)

- **Kâr amacı gütmeyen** (akademik/araştırma) kullanım: **ücretsiz**.
- **Ticari** kullanım: Arizona Üniversitesi'nden **lisans** gerekir.
- Kaynak kodu açık değil; araştırmacılar şifreli kaynağa erişim isteyebilir.



Derste kullanım kuralı

- Derste **Tigress** ikilisi ya da eski sürüm dağıtılmaz.
- Öğrenciler güncel sürümü **resmî siteden** (`tigress.wtf`) kendileri indirir.
- Lisans koşulunu **orada** doğrular.
- Yalnız **kendi** kodunuza, kendi makinenizde uygularsınız.

Neden araç, el ile değil? (1)

El ile (9. hafta)	Araçla (Tigress)
Öğreticidir	Ölçeklenir
Hataya açık	Davranışı korur (test edilmiş)
Kaynak okunamaz	Okunur kaynak sizde

Neden araç, el ile değil? (2)

El ile	Araçla
Çeşitlendirme elle olmaz	Tohumla otomatik
Tek tip, tanınabilir	Dönüşüm+tohum ile değişken

Ama araç da sihir değil

- Tigress de kırılmazlık **vermez**, maliyet yükseltir.
- İyi bilinen bir aracın kalıpları zamanla tanınabilir.
- Bu yüzden **çeşitlendirme** ve **katmanlı savunma** (RASP, sunucu) yine şart.

Temel akış · tek dönüşüm

```
# girdi: source.c → çıktı: variant.c
tigress --Transform=Flatten --Functions=grant_access \
        --out=variant.c source.c
cc -o program variant.c
```

Temel akış · üç fikir

1. `--Transform=...` hangi dönüşüm (Flatten = düzleştirme, 9. hafta K-04)
2. `--Functions=...` hangi fonksiyonlara (yalnız hassas olanlara — maliyet kuralı)
3. Çıktı yine C; **kendi derleyicinizle** derlersiniz

Neden **--Functions** ile hedef seçiyoruz?

- Gizleme her fonksiyona uygulanırsa program **çok yavaşlar/şişer**.
- Yalnız hassas fonksiyonları (lisans, anahtar türetme, bütünlük) hedefleriz.
- Bu, 9. haftadaki "maliyet kuralının" araçtaki hâli.

"Bu fonksiyonu gizlemeli miyim?" — 1

Soru 1: Bu fonksiyon hassas mı? (lisans, anahtar, bütünlük, denetim)

- **Hayır** → gizleme; maliyeti boşa harcama.
- **Evet** → devam.

"Bu fonksiyonu gizlemeli miyim?" — 2

Soru 2: Değeri yüksek mi?

- **Orta** → EncodeLiterals + EncodeArithmetic + Flatten + AddOpaque.
- **Yüksek ve küçük** → yukarısı + Virtualize.

"Bu fonksiyonu gizlemeli miyim?" — 3

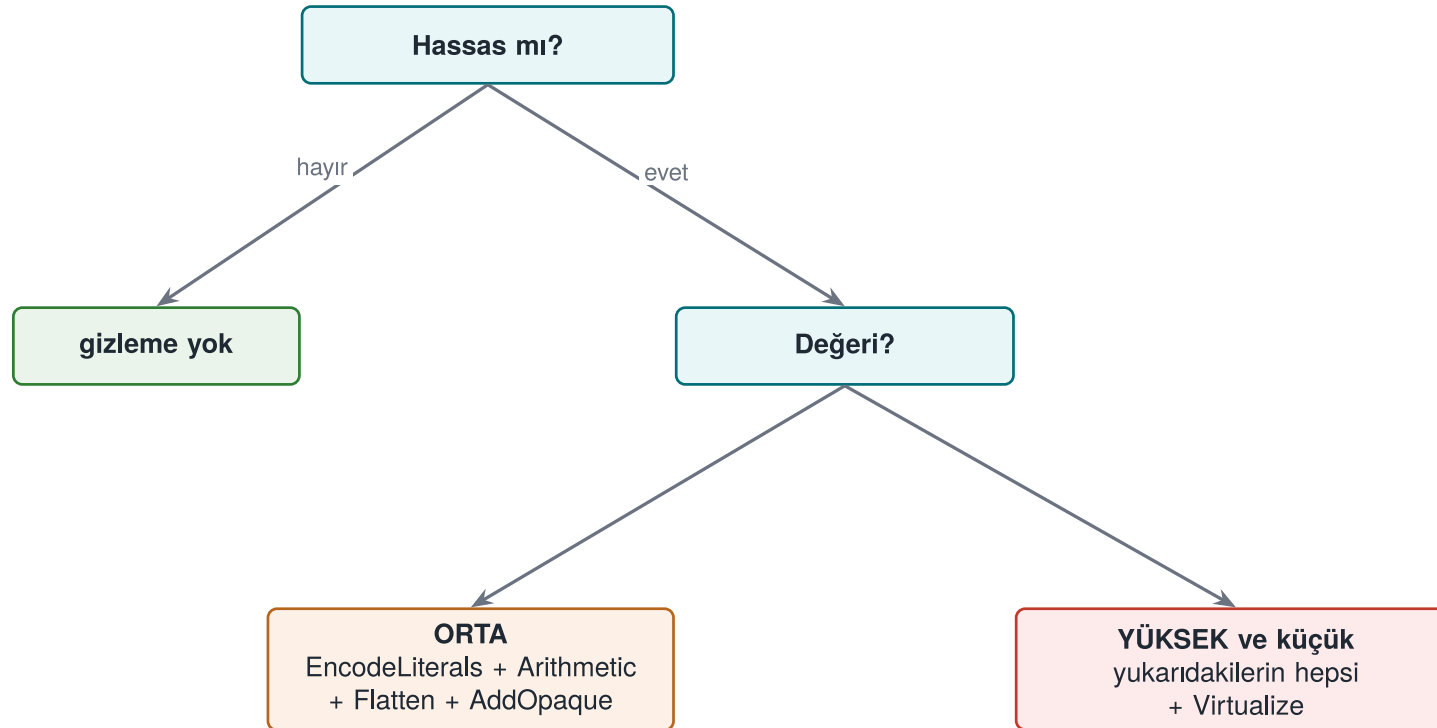
Soru 3: Her durumda:

- Çeşitlendir (tohum)
- Ölç (boyut, süre, blok)
- Birim testleriyle davranışı doğrula
- S9/S15'e yaz

Karar akışı · tek bakış

Hangi koda ne kadar gizleme? Karar akışı

önce hassas mı, sonra değeri ne kadar



Her durumda: çeşitlendir · ölç · testle · S9 ve S15 kayıtlarına yaz.

Bölüm 1 — kısa sinama

1. Kaynaktan kaynağa gizleme nedir?
2. El ile gizlemeye göre üç avantaj?
3. Tigress derste neden dağıtılmaz?

Bölüm 1 — cevaplar

1. Kaynak koda dönüşüm uygulanıp **yeni kaynak (yine C)** üretilir, sonra normal derlenir. Tigress C alır, gizlenmiş C verir.
2. **Tekrarlanabilir/otomatik** (her build), **tutarlı ve ölçekli** (çok fonksiyon), **orijinal kaynak temiz kalır** + tohumla çeşitlendirme.
3. **Lisans/kullanım koşulları**; herkes kendi indirir. Demolar Tigress yoksa **temiz türev/fallback** ile çalışır.

2. Dönüşüm aileleri ve hat

Dönüşümler = 9. hafta kuralları

Tigress dönüşümleri, 9. haftada **gördüğümüz** gizleme ailelerine karşılık gelir (K-01–K-12).

Grup grup görelim; her birini bir K-kuralına bağlayacağız.

Dönüşüm ↔ kural eşlemesi — şema

Tigress dönüşümleri ↔ 9. hafta kuralları

araç yeni bir şey icat etmiyor — aynı kuralları otomatikleştiriyor

Flatten

K-04 kontrol akışı düzleştirme

AddOpaque / InitOpaque

K-01 opak yüklem · K-03 ölü dal

EncodeArithmetic

K-02 aritmetik kodlama

EncodeLiterals

K-07 sabit / dize gizleme

Virtualize

K-10 sanallaştırma (en pahalı)

Hangi dönüşümü seçtiğinizi S9'da kuralla eşleyerek gerekçelendirin.

Kontrol akışı dönüşümleri

- **Flatten:** kontrol akışını düzleştirir → **K-04**
- **InitOpaque / AddOpaque / UpdateOpaque:** opak yüklem + sahte dal → **K-01, K-03**
- Sıra: önce **InitOpaque**, sonra **AddOpaque**.

Veri dönüşümleri

- **EncodeArithmetic:** aritmetiği denk karmaşık ifadeyle (MBA) → **K-02**
- **EncodeLiterals:** sabit ve dizeleri (string) kodlar (ucuz, neredeyse her zaman uygulanır) → **K-07, K-08**
- **EncodeData:** değişken gösterimini kodlar → **K-09**

Fonksiyon dönüşümleri

- **Split / Merge:** fonksiyonları böler/birleştirir → **K-06**
- **Virtualize:** fonksiyonu özel VM bayt koduna → **K-10**
- **Jit:** kodu çalışma anında üretir; OS korumalarıyla dikkatli kullanılmalı → **K-12 (dinamik)**

Analiz-önleme dönüşümleri

- **AntiBranchAnalysis / AntiAliasAnalysis / AntiTaintAnalysis:** statik analiz tekniklerini zorlaştırır
→ önleyici aile

Çeşitlendirme dönüşümleri

- **RandomFuns:** rastgele sahte fonksiyonlar
- **RndArgs:** sahte parametreler
- Tohumla birleşince her yapı farklı → çeşitlendirme, **K-06**

Uzayda/zamanda çeşitlendirme — şema

Uzayda ve zamanda çeşitlendirme

ikisi birlikte saldırının yeniden kullanımını kırar

UZAYDA

her kullanıcı/kopya farklı
bir kırık TEK kopyayı açar
ölçeklenemez saldırı

ZAMANDA

her sürüm farklı
bir kırık KALICI olmaz
saldırgan tekrar başlar

Çeşitlendirme tek kopyanın gücünü artırmaz; saldırının ÖLÇEKLENMESİNİ engeller.

Dönüşüm ↔ kural · tablo

Tigress	Ne yapar	9. hafta
Flatten	Düzleştirme	K-04
AddOpaque	Opak yükleme/sahte	K-01, K-03
EncodeArithmetic	Aritmetik kodlama	K-02
EncodeLiterals	Sabit/dize	K-07, K-08
Virtualize	Sanallaştırma	K-10
RandomFuns/RndArgs	Çeşitlendirme	K-06

Aileler ve maliyet

- **Ucuz:** EncodeArithmetic, EncodeLiterals
- **Orta:** Flatten, opak yüklemeler
- **Pahalı:** Virtualize (onlarca kat yavaşlama)

Bu yüzden Virtualize yalnız **küçük, kritik** fonksiyonlara; `--Functions` ile hedeflenir.

Animasyon · Bir dönüşüm, adım adım

ÖNCE — düzleştirilmemiş

durum: 9999

GRANTED

SONRA — Flatten uygulanmış

case 9999: döngü biter, aynı sonuç döner — davranış korundu.

Animasyon · Uç durum: tohum 0

GERÇEK KOD — `fallback_transform.py`

$$q = 0$$

$$(q * q) \% 4 \neq 2$$

Kanıt: tam kare mod 4 yalnız 0 ya da 1 olabilir (Collberg-Thomborson-Low 1997) — asla 2 değil.

$(0 * 0) \% 4 = 0$ — hiçbir zaman 2 değil, öyleyse yüklem HER q için doğru; dal her zaman aynı yönden geçer, ama q her tohumda farklı bayt üretir.

Dönüşümleri sırayla birleştirmek

"Tek teknik değil, birlikte"

9. haftanın en önemli kuralı.

Tigress'te bu, dönüşümleri **sırayla** uygulamak demektir.

Kavramsal hat

```
tigress \  
  --Transform=EncodeLiterals    --Functions=grant_access \  
  --Transform=EncodeArithmetic --Functions=grant_access \  
  --Transform=Flatten           --Functions=grant_access \  
  --Transform=AddOpaque         --Functions=grant_access \  
  --out=variant.c source.c
```

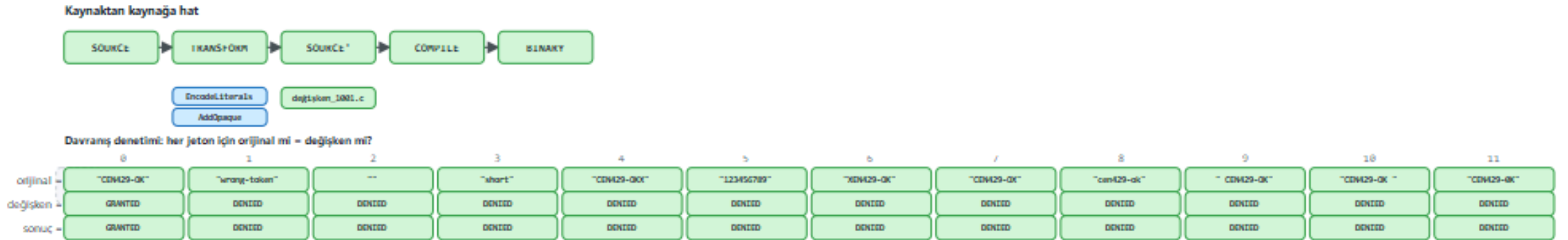
Hat ne yapıyor?

Tek denetimi (`grant_access`):

1. Sabit/dize kodla
2. Aritmetiğini kodla
3. Düzleştir
4. Opak yüklemelerle güçlendir

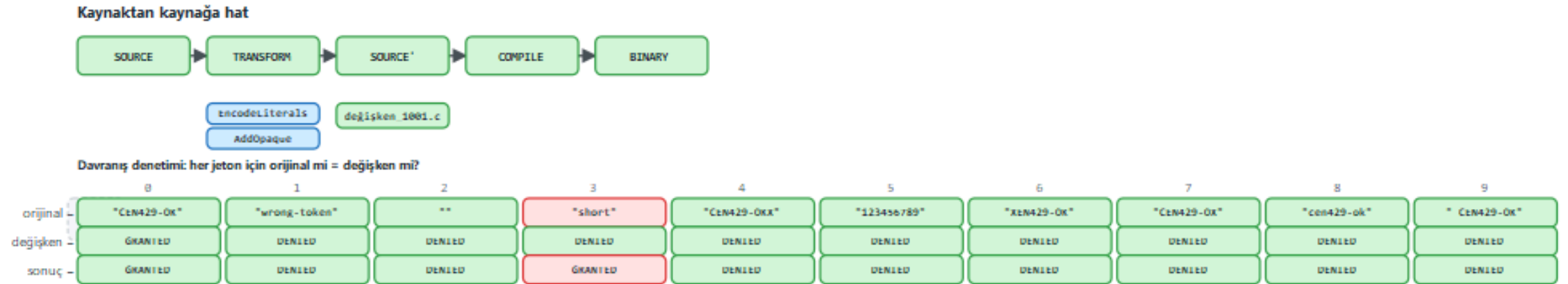
= 9. haftada K-04'ün "güçlendirilmiş düzleştirme" dediği şeyin otomatik hâli.

Animasyon · Kaynaktan kaynağa hat



12/12 jeton eşleşti: davranış korundu.

Animasyon · Uç durum: bozulmuş değişken



UYARI: en az bir jeton farklı sonuç verdi — bu değişken KABUL EDİLMEMELİ (pipeline_check.py böyle bir durumda FAIL verir).

Sıra ve test kuralı

- Dönüşüm sırası sonucu **etkiler**; kimi sıra başarımı gereksiz bozar.
- **Kural**: her hattan sonra **birim testlerini** çalıştır (davranış korunmuş mu?) ve boyut/hız ölç.
- Gizleme davranışı **asla** değiştirmemeli; değiştiriyorsa hat yanlıştır.

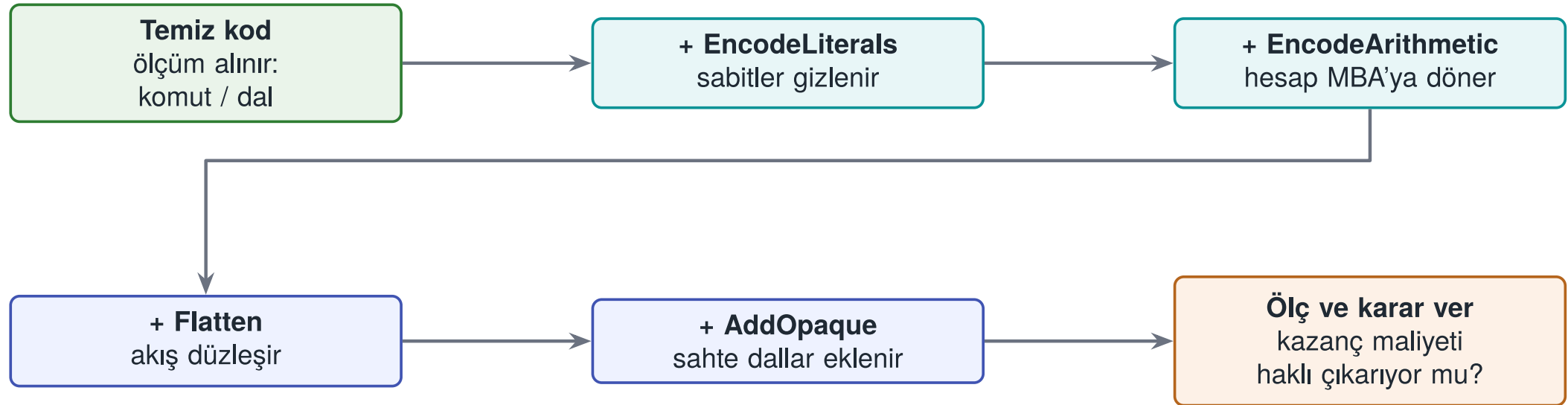
Bu, 12. haftadaki "S16 sonuçları"na bağlanır.

Adım adım örnek

Adım adım güçlendirme — şema

Bir denetimi adım adım güçlendirmek

her adımdan sonra ÖLÇ; körlemesine üst üste yığma



Maliyet katlanarak artar; kazanç bir noktadan sonra düzleşir. Durma noktasını ölçüm söyler.

Başlangıç · source.c

```
int grant_access(const char *token) {  
    if (token_is_valid(token)) return GRANTED; /* tek dal */  
    return DENIED;  
}
```

Korumasız: tek dal, `strings`, tek bayt yamasıyla atlanır.

Adım 0 · saldırgan

- `strings` → `GRANTED` / `DENIED`
- Tersine derle → tek `if`
- Başarısızlık dalını başarıya çevir

Süre: dakikalar.

Katman katman

Adım	Dönüşüm	Ne değişir	Maliyet
0	—	Tek bayt yamasıyla atlanır	—
1	EncodeLiterals	GRANTED/DENIED düz görünmez	Çok düşük
2	EncodeArithmetic	Karşılaştırma karmaşık	Düşük
3	Flatten	Tek dal görünmez	Orta
4	AddOpaque	Sahte dallar	Orta
5	Virtualize (gerekirse)	VM bayt kodu	Yüksek

Çıkarım

- Her adım saldırıyı biraz daha **pahalı** yapar.
- Her adım bir **maliyet** ekler.
- Nerede duracağınız: **varlık değeri** + **ölçtüğünüz maliyet**.

Adım 5 (Virtualize) çoğu denetim için gereksiz.

Animasyon · Sıranın etkisi: kapsam ve imza

Dönüşüm hattı (sıra soldan sağa)



Görünür sabit sayısı:

3

imza: GEÇERLİ

sign SON adım: imza son bayt içeriğiyle eşleşiyor — GEÇERLİ.

Animasyon · Uç durum: imza en başta

Dönüşüm hattı (sıra soldan sağa)



Görünür sabit sayısı:

3

imza: GEÇERSİZ (reddedildi)

signden SONRA en az bir aşama baytları değiştirdi — imza artık GEÇERSİZ (bölüm 4'teki "imzalama sırası bozulursa" senaryosu).

Bu tablo → S9'un ta kendisi

Bu "adım / dönüşüm / ne değişir / maliyet" tablosu,
projede **S9 koruma tablonuzun** doğrudan karşılığıdır.

O-LLVM (derleyici tabanlı)

- **Obfuscator-LLVM:** dönüşümleri **derleme sırasında** uygular.
- Kaynaktan kaynağa değil; LLVM ara temsili üstünde.
- ix. hafta K-11. Tigress'e bir alternatif.

Tigress mi, O-LLVM mi?

	Tigress	O-LLVM
Nerede	Kaynak (C→C)	Derleyici (IR)
Görünürlük	Gizli kaynağı görebilirsin	Derleme adımı
Dönüşüm zenginliği	Çok (Virtualize dahil)	Daha sınırlı

Ortak sınır

- İkisi de iyi bilinir → kalıpları tanınabilir.
- İkisi de **çeşitlendirme** ve **katmanlı savunma** ister.
- İkisi de kırılmazlık **vermez**.

Bölüm 2 — kısa sinama

1. Flatten, EncodeArithmetic, Virtualize hangi K-kurallarına karşılık?
2. Neden `--Functions` ile hedef seçilir?
3. Her hattan sonra **mutlaka** yapılacak iki şey?

Bölüm 2 — cevaplar

1. **Flatten** = kontrol akışı düzleştirme; **EncodeArithmetic** = aritmetik/veri gizleme; **Virtualize** = sanallaştırma (en güçlü, en pahalı).
2. Her fonksiyonu gizlemek maliyeti/performansı **patlatır**; yalnız **hassas** fonksiyonları seçmek dayanıklılığı oraya yoğunlaştırır.
3. (1) **Davranışı test et** (birim testi geçmeli), (2) **maliyeti ölç** (boyut/hız/komut sayısı, önce-sonra).

3. Çeşitlendirme ve ölçme

Kural 2'yi hatırla

Bir kopyada işe yarayan saldırı, bütün kopyalarda işe yaramasın.

Saldırgan bir kopyayı kırıp yamayı **herkese** dağıtabiliyorsa, bir kırık her yeri açar.

Çeşitlendirme nedir?

Aynı kaynaktan **davranışça eş, yapıca farklı** binary dosyalar.

Tigress bunu bir **tohum** ile yapar:

- Aynı dönüşümler + farklı tohum → opak yüklem, sahte dal, durum değerleri **değişir**.

iki tohum, iki farklı binary

```
tigress --Seed=1001 --Transform=Flatten --Transform=AddOpaque \  
--Functions=grant_access --out=gizli_a.c source.c  
tigress --Seed=2002 --Transform=Flatten --Transform=AddOpaque \  
--Functions=grant_access --out=gizli_b.c source.c
```

`gizli_a` ve `gizli_b` aynı işi yapar; makine kodları **farklı**.

İki tür çeşitlendirme

- **Uzayda:** farklı kullanıcı/cihaza farklı tohumlu kopya → bir kopyanın saldırısı diğerinde çalışmaz.
- **Zamanda:** her sürüm yeni tohum → eski saldırı yeni sürümde bozulur.

10. haftadaki anahtar/sürüm yenilemeyle birlikte çalışır.

Animasyon · Tohumla çeşitlendirme

Aynı kaynak, farklı tohumlar

tohum 1001:



tohum 2002:



tohum 3003:



Her tohum çifti farklı bayt üretti; davranış (GRANTED/DENIED) hepsinde aynı kalır — yalnız kodlama değişir.

Animasyon · Uç durum: unutulmuş tohum

Aynı kaynak, farklı tohumlar

tohum 1001:



tohum 1001:



UYARI: en az bir tohum çifti AYNI baytları üretti — tohum yanlışlıkla değiştirilmemiş olabilir (bölüm 9'daki sürüm-başı-tohum kuralını hatırlayın).

Çeşitlendirme neyi engeller?

- Gizlemenin **gücünü** artırmaz (bir kopyayı kırmak yine mümkün).
- Ama **ölçeklenmesini** engeller.

`RandomFuns` , `RndArgs` de tohumla birleşince yapıyı değiştirir.

Çeşitlendirme etkinliği · ölçüt

- Aynı kaynağı iki tohumla derle.
- Korunan fonksiyonların **bayt farkını** ölç.
- Fark ne kadar yüksekse, bir saldırının diğerinde çalışma olasılığı o kadar düşük.

Ölçütü S9'a yaz.

Gizlemeyi ölçmek

Tigress'in öğretim değeri

9. haftada gizlemeyi dört boyutta (güç, dayanıklılık, gizlilik, maliyet) ölçmeyi öğrendik.

Tigress bu ölçümleri **somut** yapmanızı sağlar: aynı programı gizle, önce/sonra ölç.

Maliyet ölçümü (kolay, zorunlu)

Her hat için ölç ve S9/S15'e yaz:

Ölçüt	Nasıl
Binary boyut	<code>size / ls -l</code>
Çalışma süresi	Aynı girdiyle N kez, ortalama
CFG karmaşıklığı	Tersine derleyicide temel blok sayısı

Maliyet ölçümü · komut

```
size ./program_temiz ./program_gizli # boyut farkı  
# süre: aynı girdiyle N kez çalıştır, ortalamayı karşılaştır
```

Basit ama **kanıt** üretir.

Maliyet · örnek tablo (Hat A ve Hat B)

Hat A: EncodeLiterals + EncodeArithmetic (hafif) · **Hat B:** Hat A + Flatten + AddOpaque (ağır)

Ölçüt	Temiz	Hat A	Hat B
Boyut	100 KB	103 KB	131 KB
Süre	1.00×	1.05×	1.80×
Blok (hedef fn.)	5	7	40
Hassas dize	3	0	0

Rakamlar örnek; siz **kendi** ölçtüğünüzü yazarsınız.

Maliyet · ham ölçüm çıktısı

```
program_temiz:  
  boyut: 100 KB  
  işlem süresi: 1.00x (referans)  
  grant_access temel blok: 5
```

```
boyut: 103 KB    (+%3)  
süre:  1.05x  
blok:  7  
strings hassas dize: 0 (öncesi 3)
```

```
boyut: 131 KB    (+%31)  
süre:  1.80x  
blok:  40
```

Animasyon · Ölçme: boyut, süre, komut, benzerlik

	orijinal (tohum 0)	tohum 1001	tohum 2002
instructions	28	51	51
jumps/calls	3	6	6
size (bytes, example)	9000 B	10666 B	10577 B
similarity to original	100%	0%	0%

Sonuç: maliyet (komut/dal) tohumdan bağımsız; boyut ve benzerlik tohuma bağlı.

Animasyon · Uç durum: iki "tohum" de orijinal

	orijinal (tohum 0)	tohum 0	tohum 0
instructions	28	28	28
jumps/calls	3	3	3
size (bytes, example)	9000 B	9000 B	9000 B
similarity to original	100%	100%	100%

Sonuç: maliyet (komut/dal) tohumdan bağımsız; boyut ve benzerlik tohumla BAĞLI DEĞİL (beklenmez).

Dayanıklılık ölçümü (ileri, kavram)

- Dayanıklılık = otomatik araca (sembolik yürütme) direnç.
- ix. hafta: "iddia değil, ölçülen bir nicelik".
- Tigress bu araştırmanın **merkezinde**.

Banescu vd. · Tigress + KLEE

Tigress dönüşümlerinin **sembolik yürütmeye** ne kadar dayandığını ölçen çalışma. Bulgular:

- Tek dönüşüm (yalnız Flatten) → çoğu zaman geri açılır.
- Dönüşümleri **birleştirmek** + durum uzayını büyütmek → çözücüyü üstel zorlar.
- Ama **maliyet** de artar.

Dayanıklılık ↔ maliyet — şema

Banescu'nun kuralı: dayanıklılık ↔ maliyet

KLEE ile ölçülen deneysel sonuç

Güçlü dönüşüm

Virtualize, çok katman

dayanıklılık YÜKSEK
performans/boyut CEZASI yüksek

Hafif dönüşüm

EncodeLiterals, opak yüklem

dayanıklılık düşük
maliyet düşük

Koruma, varlığın değeriyle ORANTILI seçilir — her şey en güçlüyle gizlenmez.

Ders için çıkarım

Dayanıklılık ve maliyet **ödünleşir** (9. haftadaki gibi).

"Güçlü" deme; "**şu araca karşı şu boyutta problemi şu sürede çözmedi**" de.

Ölçüm kuralı (S9/S15)

Bir gizleme kararını ölçüyle savun:

"Flatten + AddOpaque hattı boyutu %30 büyüttü, işlemi $1.8\times$ yavaşlattı; hedef fonksiyonun temel blok sayısı 8 katına çıktı."

Ölçülmemiş gizleme = değerlendiricide (12. hafta) **iddia**.

Karar

- Varlık değeri **düşük** → Hat A yeter.
- Varlık değeri **yüksek** → Hat B (maliyeti kabul et) + çeşitlendirme.

Karar sayılara dayanır, hisse değil.

Çeşitlendirme ölçümü (örnek)

- Hat B'yi tohum 1001 ve 2002 ile üret.
- Korunan fonksiyonun bayt farkı: %92.
- Yorum: bir kopyaya yazılan yama diğerinde büyük olasılıkla çalışmaz.

Bölüm 3 — kısa sinama

1. Tohum çeşitlendirmeyi nasıl sağlar?
2. Uzayda ve zamanda çeşitlendirme farkı?
3. Banescu vd.'nin ana kuralı? (dayanıklılık ↔ maliyet)

Bölüm 3 — cevaplar

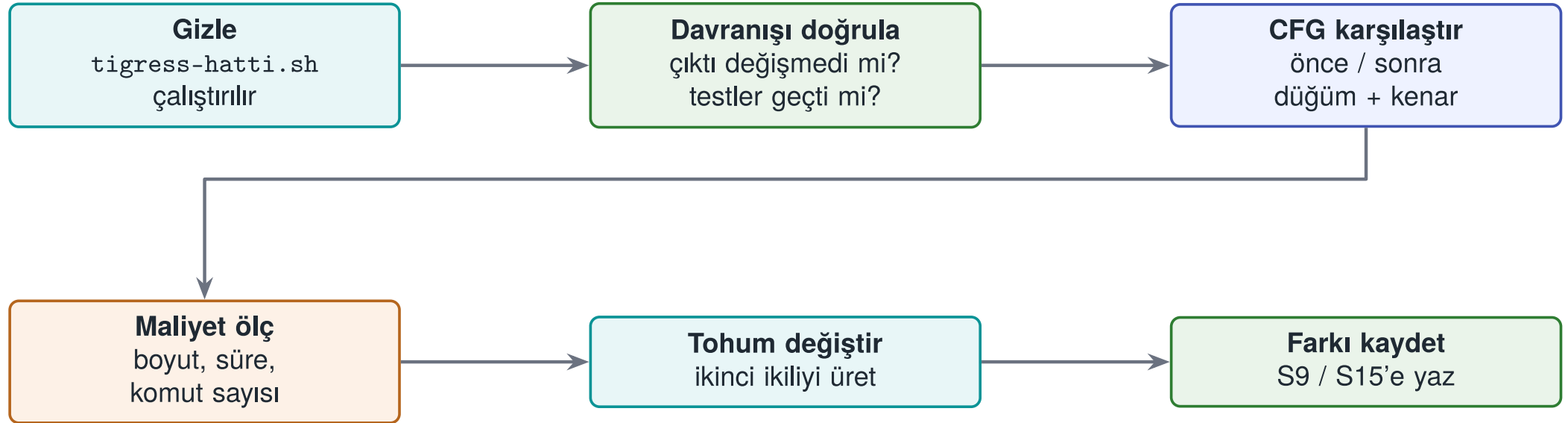
1. Aynı kaynağı farklı **tohum**la gizleyince araç farklı rastgele seçimler yapar → her build **farklı binary** (aynı davranış).
2. **Uzayda**: farklı kopyalar/kullanıcılar farklı (bir kırık herkesi kırmaz). **Zamanda**: sürümler arası değişir (kırık kalıcı olmaz).
3. Daha güçlü gizleme daha çok **dayanıklılık** ama daha çok **maliyet**; koruma varlığın **değeriyle orantılı** seçilir.

4. Sınıf içi akış (adım adım)

Sınıf içi akış — şema

Sınıf içi akış: gizle, karşılaştıır, ölç, çeşitlendir

ölçmediğiniz korumayı savunamazsınız



Davranış doğrulaması atlanırsa gizleme sessizce programı bozar — en sık yapılan hata budur.

Ne yapacağız?

Kendi küçük, sentetik programınızı:

gizle → davranışı doğrula → karşılaştır → ölç → çeşitlendir → raporla

Adım adım gidelim.

Adım 1 · Hazırlık

- Küçük, sentetik bir program yaz (ör. `grant_access`).
- Birim testleriyle **doğru çalıştığını** göster.
- Bu senin "temiz" temel çizgin.

Adım 2 · Gizle

```
tigress --Transform=EncodeLiterals --Transform=EncodeArithmetic \  
        --Transform=Flatten --Transform=AddOpaque \  
        --Functions=grant_access --out=variant.c source.c  
cc -o program_gizli variant.c
```

Bir dönüşüm hattı uygula.

Adım 3 · Davranışı doğrula

- Aynı birim testlerini **gizli** sürümde çalıştır.
- Sonuç **birebir aynı** olmalı.
- Farklıysa: hat yanlış → düzelt.

Gizleme davranışı asla değiştirmez.

Adım 4 · Karşılaştır

- Tersine derleyicide (Ghidra/objdump) temiz ve gizli sürümü aç.
- Kontrol akışını karşılaştır.
- Temel blok sayısındaki **artışı** not et.

Adım 5 · Ölç

```
size ./program_temiz ./program_gizli  
# süre: aynı girdiyle N kez çalıştır, ortalamayı karşılaştır
```

Boyut ve süre farkını yaz.

Adım 6 · Çeşitlendir

- Aynı hattı **iki farklı tohumla** çalıştır.
- İki binary'nin farklı olduğunu göster (bayt farkı).

```
tigress --Seed=1001 ... --out=gizli_a.c source.c  
tigress --Seed=2002 ... --out=gizli_b.c source.c
```

Adım 7 · Raporla

Bulguları bir tabloya dök:

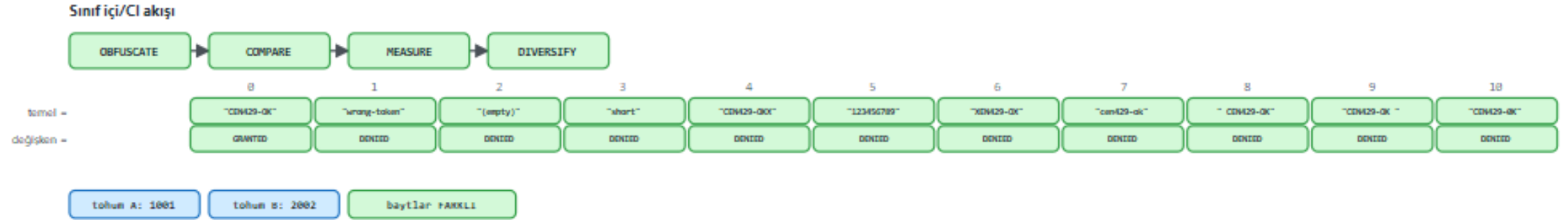
Dönüşüm	Boyut	Süre	Blok	Tohum farkı
...	...	...	...	...

Bu tablo doğrudan **S9/S15**'e girer.

Etik ve güvenli kullanım

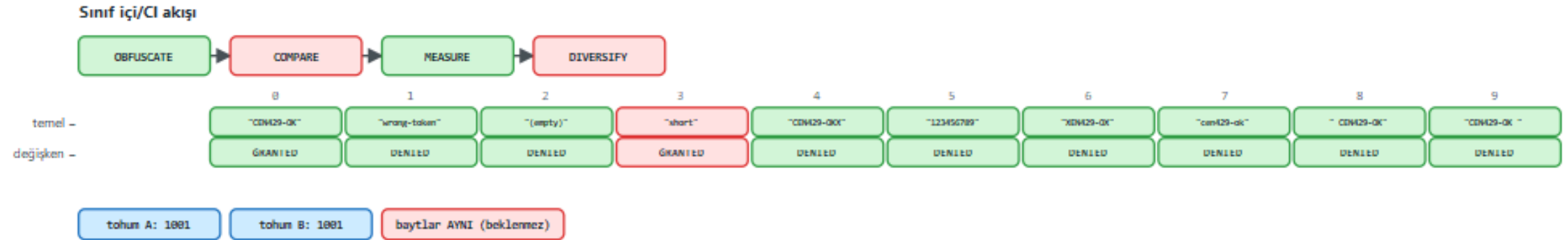
- Tigress yalnız **kendi** kodunuza.
- Örnek program bilgisayara **zarar vermez**: sistem ayarı değiştirmez, ağa dokunmaz.
- Bütün değerler **sentetik**.

Animasyon · Sınıf içi akış



PIPELINE CHECK PASSED: davranış korundu, çeşitlendirme çalışıyor.

Animasyon · Uç durum: uyuşmazlık + tekrar tohum



PIPELINE CHECK FAILED: yukarıdaki adım nedeniyle akış burada durur.

Sınıf içi · özet akış

Her hattan sonra mutlaka iki şey

ölçmeden “güçlendirdim” denemez



Örnek ölçüm: `erisim_ver` 28 komut/3 dal → 51 komut/6 dal.

Adım adım akış · neden bu sıra?

- Önce **davranış** (testler) — bozulmadığından emin ol.
- Sonra **kanıt** (CFG, ölçüm) — kazancı göster.
- Sonra **çeşitlendirme** — ölçeklenmeyi kır.

Ölçmeden "güçlü" deme.

Yeni örnek · bütünlük denetimi

```
int dosya_saglam(const uint8_t *veri, size_t n,
                 const uint8_t *beklenen) {
    uint8_t ozet[32];
    ozet_hesapla(veri, n, ozet);          /* SHA-256 benzeri */
    return sabit_zamanli_esit(ozet, beklenen, 32);
}
```

Sürüm bütünlüğünü denetleyen sentetik bir fonksiyon.

Neden bunu gizleyelim?

- Saldırgan bu denetimi bulup **atlatmak** ister (kurcalanmış dosyayı geçirmek için).
- Denetimin dönüş noktası ve karşılaştırması hedef.
- ix. hafta: opak boolean + rastgele çıkış burada kritik.

Hat seçimi

```
tigress \  
  --Transform=EncodeLiterals \  
  --Transform=EncodeArithmetic \  
  --Transform=Flatten \  
  --Transform=AddOpaque \  
  --Functions=dosya_saglam \  
  --out=variant.c source.c
```

Adım adım · ne olur?

- EncodeLiterals: 32 ve varsa dizeler düz görünmez.
- EncodeArithmetic: karşılaştırma karmaşıklaşır.
- Flatten: tek dönüş noktası görünmez.
- AddOpaque: sahte dallar; "başarı" dalı tek yerde değil.

Davranış doğrulama (zorunlu)

- Doğru özetli dosya → **geçer**.
- Bozuk özetli dosya → **reddedilir**.
- Gizli sürümde de aynı sonuç → hat doğru.

CFG önce/sonra (kavram)

Kontrol akışı: düzleştirmeden önce ve sonra

aynı sonucu üreten iki program, çok farklı iki grafik

ÖNCE — okunur

[hesapla] → [karşılaştır]
→ [geç] / [red]
birkaç blok
akış gözle izlenir

SONRA — düzleştirilmiş

[switch] ↔ {çok sayıda case}
+ sahte durumlar
çok blok
sıra durum değişkeninde saklı

Amaç: blok sayısını ve dallanmayı artırıp statik okumayı pahalı kılmak.

Denetimin nerede geçtiği/kaldığı akıştan okunmaz.

Sabit zamanlılık korunmalı

- `sabit_zamanli_esit` gizlense de **sabit zamanlı** kalmalı.
- Gizleme uğruna erken çıkış eklenirse yan kanal açılır (3. hafta).
- Dönüşümlerin bunu bozmadığını **test et**.

Ölçüm (örnek)

Ölçüt	Önce	Sonra
Boyut	100 KB	128 KB
Süre	1.0×	1.7×
Blok (dosya_saglam)	4	33

Çeşitlendirme

- İki tohumla üret → iki farklı binary.
- "Bütünlük denetimini atla" yaması bir kopyada işe yarasa bile diğerinde yaramaz.

İkinci örnek · çıkarım

- Bütünlük denetimi gibi **atlatma hedefi** fonksiyonlarda gizleme değerlidir.
- Ama asıl güç: gizleme + RASP (6. hafta) + sunucu denetimi.
- Gizleme tek başına atlatmayı **geciktirir**, engellemez.

Bölüm 4 — kısa sinama

1. Gizledikten sonra ilk **mutlaka** yapılacak şey?
2. Çeşitlendirmeyi nasıl gösterirsin?
3. Rapor tablosu hangi proje bölümlerine gider?

Bölüm 4 — cevaplar

1. Davranışın **değişmediğini test et** (birim testi geçmeli); gizleme işlevi bozmamalı.
2. İki farklı tohumla üret; binary'leri karşılaştır (**hash/boyut/komut farklı**) ama **aynı girdi→aynı çıktı** (diff/objdump).
3. **S9** (kod sağlamlaştırma (hardening)) ve **S15** (derleme/dağıtım hattı); önce/sonra ölçüm kanıt olarak.

5. Derleme hattı (S15) ve proje

Gizleme "en sonda elle" değil

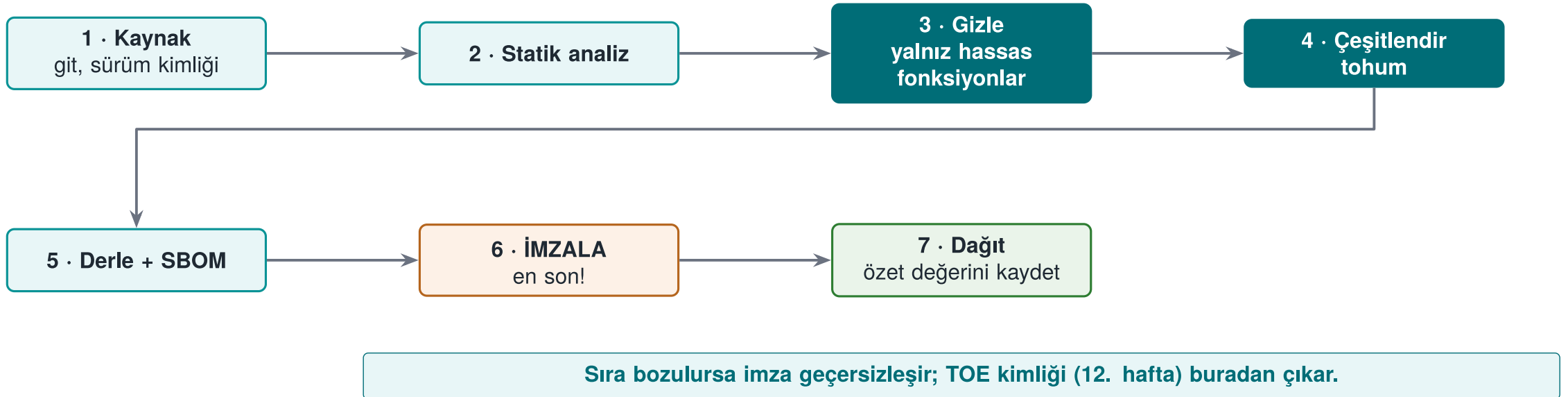
Gizleme, derleme **hattının** bir adımı olmalı.

Vize sonrası projenizin **S15** bölümü tam bunu belgeler.

Derleme hattı · şema

S15: derleme ve dağıtım hattı

imza en sonda: önce gizle, sonra imzala



Hat kuralı 1 · yalnız hassas fonksiyonlar

- `--Functions` ile yalnız hassas fonksiyonları gizle.
- Gerekçesini yaz (hangi fonksiyon, neden).
- Maliyet kuralı (9. hafta).

Hat kuralı 2 · her sürüm çeşitlendirilir

- Her sürüm bir **tohumla** çeşitlendirilir.
- Tohum ve sürüm kimliği **kaydedilir**.
- Zamanda çeşitlendirme (bölüm 3).

Hat kuralı 3 · imzalama gizlemeden sonra

- Önce gizle, **sonra** imzala.
- Sürüm kimliği ve özet değerleri (12. haftadaki TOE) **tutarlı** olsun.
- Neden sonra? İmza, dağıtılan **son** binary'yi kapsamalı.

Hat kuralı 4 · CI'da otomatik

- Hat, sürekli entegrasyonda (CI) çalışır.
- Her sürümde: birim testleri + boyut/hız ölçümü **otomatik**.
- Davranış bozulursa yapı **durur**.

Animasyon · CI hattı, sürüm başına tohum

Her sürüm: kendi tohumuyla bir CI çalışması

sürüm v1.0 (tohum 1001):



sürüm v1.1 (tohum 4832):



sürüm v1.2 (tohum 9214):



3/3 sürüm başarıyla yayınlandı; hiçbir tohum tekrarlanmadı.

Animasyon · Uç durum: tekrarlanan tohum

Her sürüm: kendi tohumuyla bir CI çalışması

sürüm v1.0 (tohum 1001):



sürüm v1.1 (tohum 4832):



sürüm v1.2 (tohum 9214):



sürüm v1.3 (tohum 1001):



En az bir sürüm CI'yi geçemedi — yukarıdaki adımlara bakın.

9, 11, 14 birlikte

- **9:** gizleme kuralları (kod)
- **11:** whitebox (anahtar)
- **14:** otomasyon + çeşitlendirme (bu hafta)

Ortak kural: koruma = **gecikme**; güç katmanlardan ve ölçümden.

Proje · S9 ileri + S15 hat — 1

1. Dönüşüm hattı (S9): hangi fonksiyonlar, hangi dönüşümler, hangi sırayla, **neden**.

Proje · S9 + S15 — 2

2. Ölçüm tablosu: boyut, süre, (varsa) blok sayısı — önce/sonra.

Proje · S9 + S15 — 3

3. **Çeşitlendirme:** iki tohumla üretim + fark ölçütü; uygulamayacaksanız **gerekçe**.

Proje · S9 + S15 — 4, 5

4. **S15 hattı:** gizleme + imzalama + sürüm kimliği adımlarını **diyagramla** belgele.

5. **Davranış kanıtı:** gizli sürümün birim testlerini geçtiğini göster (S16 bağı).

Değerlendirici gözüyle

- "Çok dönüşüm kullandım" **değil**.
- **Her dönüşümün gerekçesi ve ölçüsü.**
- Ölçülmemiş gizleme = iddia (12. hafta).

Vaka · kurgu

Sentetik bir uygulamada bir **lisans denetimi** ve bir **anahtar türetme** fonksiyonu var.

İkisini de mi ağır gizleyeceğiz?

Vaka · karar

- **Lisans denetimi:** orta değer → Hat B (Flatten + AddOpaque).
- **Anahtar türetme:** yüksek değer → Hat B + **Virtualize** (küçük olduğu için maliyet kabul edilebilir).
- Gerisi: gizlenmez (maliyet gereksiz).

Vaka · çeşitlendirme + hat

- Her sürüm farklı tohum.
- Derleme hattı: gizle → imzala → sürüm kimliği.
- CI'da birim testleri + ölçüm otomatik.

Vaka · S9/S15 çıktısı

Fonksiyon	Hat	Boyut/süre	Gerekçe
lisans_dogrula	B	+%31 / 1.8×	orta değer
anahtar_turet	B+Virtualize	+%60 / 3×	yüksek değer, küçük fn.

Vaka · kalan risk

- Yeterince kararlı saldırgan tek kopyada çözebilir.
- Telafi: çeşitlendirme + kısa ömürlü anahtar + sunucu denetimi.
- **Açıkça yazılır.**

Bölüm 5 — kısa sinama

1. İmzalama neden gizlemeden **sonra**?
2. "Yalnız hassas fonksiyonlar" kuralının gerekçesi?
3. S15 hattının adımları?

Bölüm 5 — cevaplar

1. İmza **son binary'yi** korur; önce imzalayıp sonra gizlersen binary değişir → imza geçersiz. Sıra: derle → gizle → **imzala**.
2. Bütünü gizlemek performans/boyutu patlatır ve çoğu kod hassas değil; korumayı **kritik** fonksiyonlara yoğunlaştır → maliyet düşük, etki yüksek.
3. Kaynak → (statik analiz) → derle → **gizle** → çeşitlendir (tohum) → pakitle/SBOM → **imzala** → dağıt (hepsi CI'da kayıtlı).

Çözümlü kendini sinama

Soru 1

Kaynaktan kaynağa gizleme nedir? El ile gizlemeye göre üç avantajı?

Cevap: Araç C kaynağı alıp gizli C kaynağı üretir. Avantajlar: davranışı korur (hataya kapalı), okunur kaynak sizde kalır, tohumla çeşitlendirme kolay.

Soru 2

Tigress'in dersteki yeri; hangi lisans, neden derste dağıtılmaz?

Cevap: 9. hafta kurallarının otomatik hâli. Kâr amacı gütmeyen kullanım ücretsiz, ticari için Arizona lisansı. Kaynak kapalı → ikilisi derste dağıtılmaz; öğrenci resmî siteden indirir.

Soru 3

Şu dönüşümleri 9. hafta kurallarıyla eşle: Flatten, EncodeArithmetic, EncodeLiterals, Virtualize, AddOpaque.

Cevap: Flatten→K-04, EncodeArithmetic→K-02, EncodeLiterals→K-07/K-08, Virtualize→K-10, AddOpaque→K-01/K-03.

Soru 4

Bir dönüşüm hattı neden tek dönüşümden güçlü? Sıra neden önemli?

Cevap: Katmanlar birbirini güçlendirir (9. hafta "birlikte"). Sıra sonucu ve başarımı etkiler; kimi sıra gereksiz yavaşlatır ya da zayıf sonuç verir.

Soru 5

Her dönüşüm hattından sonra hangi iki şeyi mutlaka yaparsın?

Cevap: (1) Birim testlerini çalıştır (davranış korunmuş mu?), (2) boyut/hız ölç (maliyet).

Soru 6

Virtualize neden yalnız küçük ve kritik fonksiyonlara uygulanır?

Cevap: Maliyeti çok yüksek (onlarca kat yavaşlama, boyut). Getirisi ancak en değerli, küçük fonksiyonlarda maliyeti haklı çıkarır.

Soru 7

Tohum (--Seed) çeşitlendirmeyi nasıl sağlar? Uzayda/zamanda fark?

Cevap: Farklı tohum → farklı opak yükleme/sahte dal/durum. Uzayda: farklı kopyalar aynı anda. Zamanda: her sürüm yeni tohum.

Soru 8

Çeşitlendirme gizlemenin gücünü mü, ölçeklenmesini mi engeller?

Cevap: Gücünü artırmaz (bir kopya yine kırılabilir); **ölçeklenmesini** engeller — bir kırık her yeri açmaz.

Soru 9

Banescu vd. çalışmasının ders için ana kuralı?

Cevap: Dayanıklılık ölçülen bir niceliktir. Dönüşümleri birleştirmek sembolik yürütmeyi üstel zorlar; ama maliyet de artar → **dayanıklılık** ↔ **maliyet ödünleşir**.

Soru 10

Gizlemeyi derleme hattına koyarken imzalama neden gizlemeden sonra?

Cevap: İmza, dağıtılan **son** binary'yi kapsamalı. Önce gizlenir, sonra imzalanır; sürüm kimliği/özet tutarlı olur.

Soru 11

S15'te "yalnız hassas fonksiyonlar gizlenir" kuralının gerekçesi?

Cevap: Her fonksiyonu gizlemek programı çok yavaşlatır/şişirir. Maliyeti yalnız değerli fonksiyonlarda üstlenmek mantıklı.

Soru 12

9, 11 ve 14. haftaların ortak kuralını bir cümleyle yaz.

Cevap: Koruma kırılmazlık değil **gecikme** sağlar; gücü **katmanların birleşiminden**, **çeşitlendirmeden** ve **ölçülmüş** olmaktan gelir.

Kapanış · üç hafta

- **9:** gizleme kuralları (el ile)
- **11:** whitebox (anahtar)
- **14:** otomasyon + çeşitlendirme

Hepsi aynı çerçeve: maliyet + katman + ölçüm.

Klasik hatalar — özet

Hata	Nerede işlendi
Her şeyi gizlemek	Bölüm 1 (--Functions), Bölüm 2 (maliyet)
Testsiz gizleme	Bölüm 2 (sıra ve test kuralı)
Ölçmeden "güçlü" demek	Bölüm 3 (ölçüm kuralı)
Çeşitlendirmeyi atlamak	Bölüm 3 (Kural 2)
İmzayı gizlemeden önce yapmak	Bölüm 5 (Hat kuralı 3)
Eski/dağıtılan Tigress kullanmak	Bölüm 1 (lisans)

Terimler sözlüğü

Terim	Anlam
Kaynaktan kaynağa	C girip gizli C çıkaran gizleme
Dönüşüm	Tek gizleme işlemi
Hat	Dönüşümlerin sıralı bütünü
Tohum	Rastgeleliği yöneten sayı (çeşitlendirme)
Sembolik yürütme	Yol-çözen otomatik analiz (KLEE)

Dönüşüm seçim kılavuzu

İstediğin	Dönüşüm
Dize/sabit gizle	EncodeLiterals
Hesabı gizle	EncodeArithmetic
Yapıyı gizle	Flatten
Sahte/opak ekle	AddOpaque
Makine kodunu gizle	Virtualize (pahalı)
Çeşitlendir	--Seed, RandomFuns

Kontrol listesi

- [] Yalnız hassas fonksiyonlar (`--Functions`)
- [] Hat sırası mantıklı (ucuzdan pahalıya)
- [] Birim testleri geçiyor (davranış)
- [] Boyut/süre/blok ölçüldü
- [] İki tohumla çeşitlendirildi
- [] İmza gizlemeden sonra
- [] S9/S15'e yazıldı

Kaynaklar

- **Tigress** resmî sitesi/çalışma sayfaları (`tigress.wtf`) — dönüşümler, sözdizimi, v4, lisans
- Collberg & Nagra, *Surreptitious Software* — taksonomi, ölçme
- Banescu vd. — Tigress + KLEE dayanıklılık ölçümü
- Obfuscator-LLVM — derleyici tabanlı alternatif

Özet: bu haftanın tek cümlesi

Tigress, 9. haftanın gizleme kurallarını **otomatik** ve **çeşitlendirilmiş** uygular; ama kural aynı: gizleme

kırılamazlık değil gecikme; birleştir, çeşitlendir, **ölç**.

Araç güçlüdür ama **karar sizindir**: neyi, neden, hangi maliyetle gizliyorsunuz?

Bir sonraki hafta

15. hafta — Final proje gösterimleri (RAP2)

Dönem içeriği tamamlandı. Final raporunda bu haftanın **hattı (S15)** ve **ölçümleri (S9)** beklenir. 16. hafta: Quiz-2.